

How Arrays Work Internally

How Arrays Work Internally

1. Random Access — What It Actually Means
2. Stack vs. Heap: Primitives vs. Non-Primitives
3. Deriving the Random Access Formula
4. Boolean: The One Type Without a Fixed Size
5. 2D Arrays: Two-Step Address Lookup
6. Strings: Also Stored via References
7. Bounds Checking: `ArrayIndexOutOfBoundsException`
8. Why Random Access Matters: CPU Caching

★ One-Page Revision

📝 Practice Questions

🔥 Quick FAQ

How Arrays Work Internally

Heap Storage, Random Access, and the Address Formula

1. Random Access — What It Actually Means

When you access `arr[3]`, the JVM doesn't loop through index `0`, `1`, `2` to reach it — it **jumps directly** to that memory location.

Definition: Random access means any array element can be fetched directly, in constant time, without scanning through the elements before it.

This lecture explains *how* the JVM pulls this off — using nothing more than simple arithmetic.

2. Stack vs. Heap: Primitives vs. Non-Primitives

Primitive types (`int`, `char`, `boolean`, etc.) store their value **directly**, in **stack memory**:

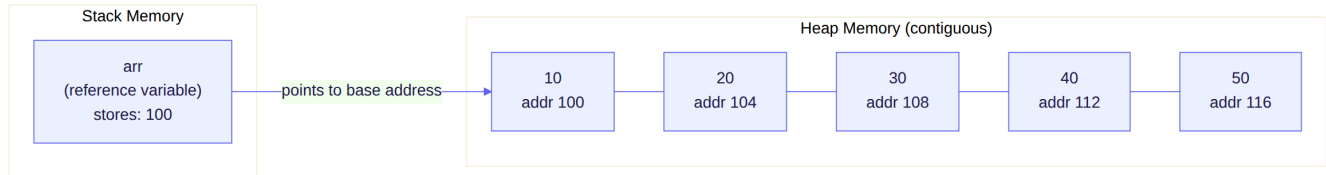
```
int x = 4;
```

`x` directly holds `4`. Simple.

Non-primitive types (arrays, and later objects/Strings) work differently. Since arrays need `new`, the actual data is allocated in a separate region called **heap memory** — and the variable itself, sitting in the **stack**, only stores a **reference** (the heap address where the real data begins):

```
int[] arr = new int[5];
```

	Where it lives	What it holds
<code>arr</code> (the variable)	Stack	A reference — the base address of the array in the heap
The actual 5 integers	Heap	The real, contiguous data



Important: `arr` does **not** directly hold the array. It's a **reference variable** — think of it as an arrow pointing at the array's starting address in heap memory. Anything created with `new` lives in the heap; anything created without `new` lives in the stack.

Easy Hinglish Explanation

"Primitive variable seedha apni value hold karta hai. Lekin array jaisi non-primitive cheez ke liye, stack mein sirf ek reference (address) store hota hai, aur asli data heap memory mein alag se padha hota hai."

3. Deriving the Random Access Formula

Since heap memory for an array is **contiguous**, each element sits at a predictable offset from the one before it. If an `int` takes 4 bytes, and the array starts at address `100`:

Index	Address
0	100
1	104
2	108
3	112
4	116

Each address is exactly `4 bytes` more than the last — because every `int` occupies exactly 4 bytes, back-to-back.

The General Formula

★ **Core Formula** — memorize this: $\text{ElementAddress} = \text{BaseAddress} + (\text{DataTypeSize} \times \text{Index})$

Worked example — finding `arr[3]`:

```
BaseAddress = 100
DataTypeSize (int) = 4 bytes
Index = 3

Address = 100 + (4 × 3) = 112
```

The JVM jumps straight to address `112`, reads 4 bytes, and returns the value stored there — no iteration required.

Important: The JVM already knows the **base address** (stored in the reference variable on the stack) and the **data type size** (fixed per type — `int` =4B, `long` =8B, `float` =4B, `double` =8B, `char` =2B). Only the index changes

per access, so this formula computes any element's address in a single step — **O(1)**.

4. Boolean: The One Type Without a Fixed Size

Every primitive type has an official, fixed byte size — **except `boolean`**.

 **Important:** According to the official Java documentation, `boolean` has **no fixed size**. Java leaves this decision to each individual JVM implementation.

Logically, a boolean only needs **1 bit** (true/false). But most real-world JVMs (like HotSpot, used by Oracle and OpenJDK) allocate a full **1 byte (8 bits)** per boolean instead.

⚠ **Common Mistake:** Assuming `boolean` takes the theoretical minimum (1 bit). In practice, it almost always takes **1 byte**.

Why waste 7 extra bits? CPUs fetch memory in **byte-sized chunks**, not individual bits. Allocating a full byte per boolean — even though only 1 bit is logically needed — lets the CPU read/write it in a single, natural fetch operation instead of doing complex bit-level extraction. This trades a little memory for a lot of speed.

Easy Hinglish Explanation

"Boolean ko sirf 1 bit chahiye hota hai logically. Lekin CPU hamesha bytes mein data fetch karta hai, bits mein nahi. Isliye JVM boolean ko poora 1 byte de deta hai — thoda memory waste hota hai, lekin CPU access bahut fast ho jaata hai."

5. 2D Arrays: Two-Step Address Lookup

A 2D array, as covered previously, is an **array of arrays** — meaning `arr` is a reference to an array of *references*, where each of those references points to its own separately-allocated row array.

```
int[][] arr = new int[3][4];
```

Since there are now **two levels of indirection**, finding `arr[i][j]` needs the formula **applied twice**:

Step 1 — find the row's base address:

```
RowAddress = BaseAddress + (ReferenceSize × i)
```

 **Important:** References themselves have a fixed size too — **4 bytes**, regardless of what they point to. So this first step uses `4`, not the element's data type size, because you're looking up an *address*, not a value yet.

Step 2 — find the element within that row:

```
ElementAddress = RowAddress + (DataTypeSize × j)
```

Worked example: Outer array base = `100`, row 1's base address (read from the outer array) = `200`. Finding `arr[1][2]`:

```
Step 1: RowAddress = 100 + (4 × 1) = 104 → read the reference stored at 104, which points to 200
Step 2: ElementAddress = 200 + (4 × 2) = 208
```

The JVM reads the reference at **104** first (getting **200**), then jumps into that row's memory and applies the formula again to land on **208**.

★ **Exam Tip:** This two-step process is exactly why 2D arrays support **jagged rows** — each row is independently allocated, so nothing forces the JVM to compute a single flat offset the way it does for 1D arrays.

6. Strings: Also Stored via References

String is another non-primitive type, so a **String[]** array works exactly like a 2D array conceptually — the array holds **references**, not the actual text:

```
String[] names = new String[3];
```

Index	Reference stored	Points to (heap)
0	100	"Aditya"
1	200	"Abhay"
2	300	"Rohit"

Finding **names[1]** uses the **same formula**, with reference size (4 bytes):

```
Address = 400 + (4 × 1) = 404
```

The JVM reads the reference at **404**, gets **200**, then jumps to address **200** in the heap and reads the actual string **"Abhay"**.

🔥 **Important:** Up through **JDK 8**, a **String** was internally implemented as a **char[]** (character array) — which is why you can visualize a string as a sequence of characters. From **JDK 9 onward**, the internal representation changed and became more complex; a **String** is no longer literally a **char[]** under the hood. Either way, this internal detail is a topic for a later, dedicated lecture on Strings.

7. Bounds Checking: **ArrayIndexOutOfBoundsException**

Before applying the address formula, the JVM checks whether the requested index is actually valid:

```
if (index < 0 || index >= arr.length) {
    throw new ArrayIndexOutOfBoundsException();
}
```

⚠ **Common Mistake:** Assuming an invalid index just returns garbage or **null**. It doesn't — Java explicitly checks bounds **every time** and throws immediately if the index is negative or **≥ length**, preventing the program from ever reading unintended memory.

8. Why Random Access Matters: CPU Caching

Random access isn't just about avoiding loops — it also plays directly into how CPUs fetch memory.

CPUs don't fetch memory one byte at a time; they fetch in larger chunks (e.g., 8 bytes at once) into a small, extremely fast on-chip memory called a **register/cache**. Because array elements are stored **contiguously**, fetching one element

often pulls the *next* element into the cache **for free**, alongside it.

Example: Fetching `arr[3]` (a 4-byte `int`) as part of an 8-byte CPU fetch also happens to pull in `arr[4]` at the same time. If the next line of code accesses `arr[4]`, the CPU already has it cached — no second trip to RAM needed.

★ **Exam Tip:** This is a major reason arrays outperform many other data structures for **sequential access patterns** — contiguous memory plus CPU cache pre-fetching means iterating through an array is significantly faster than iterating through scattered memory locations.

💡 Easy Hinglish Explanation

"CPU jab bhi memory se data fetch karta hai, to ek saath kai bytes le aata hai — na ki sirf ek. Kyunki array ka data contiguous hota hai, ek element fetch karte waqt agla element bhi free mein CPU ke cache mein aa jaata hai. Isi wajah se array pe loop chalana bahut fast hota hai."

★ One-Page Revision

Core Definitions

- **Random access** = fetching any array element directly, in $O(1)$ time, without scanning prior elements
- **Reference variable** = a stack-stored variable holding the heap address of the actual (non-primitive) data
- **Base address** = the memory address where an array's first element begins

The Formula

$$\text{ElementAddress} = \text{BaseAddress} + (\text{DataTypeSize} \times \text{Index})$$

- `int` / `float` = 4 bytes · `long` / `double` = 8 bytes · `char` = 2 bytes · reference = 4 bytes

Key Facts

- Primitives store values directly in the **stack**; non-primitives (arrays, Strings) store data in the **heap**, with a reference in the stack
- `boolean` has **no official fixed size** — most JVMs use 1 byte (not 1 bit) for CPU efficiency
- 2D arrays need the formula applied **twice** — once for the row reference, once for the element within that row
- `String[]` arrays store references to String objects, not the text itself
- Invalid indices throw `ArrayIndexOutOfBoundsException`, checked before every access
- Contiguous memory + CPU caching is why arrays are fast for sequential access



Practice Questions

Basic

1. Where are primitive values stored, and where are array elements stored?
2. Write the general random access formula.
3. What does a reference variable actually store?

Medium 4. For an `int` array with base address `200`, what's the address of `arr[5]`? 5. Why does finding an element in a 2D array require applying the formula twice? 6. Why does `boolean` not have an official fixed size in Java?

Interview 7. Explain why arrays provide $O(1)$ random access while a hypothetical non-contiguous structure would not. 8. How does CPU caching make sequential array access faster than random access to scattered memory? 9. Why do most JVMs allocate a full byte to a boolean instead of a single bit?

Quick FAQ

Does every JVM use 1 byte for boolean? Not guaranteed by the Java spec — but HotSpot (Oracle/OpenJDK), the most common JVM, does.

Is a reference the same as the actual array? No — the reference is just an address (4 bytes) pointing to where the real array data lives in the heap.

Why is array access faster than, say, a linked list? Linked lists aren't stored contiguously, so there's no fixed formula to jump to element `i` — you must traverse node by node, making access $O(n)$ instead of $O(1)$.